

Contents

Review — Alina’s Circuit 4 v3 single-final-root drop (2026-05-22)	1
TL;DR	1
1. Что Алина изменила в circuit’е	2
2. Что Алина построила в prover-репо	4
3. Что меняется у нас на ETH-стороне (Phase B implementation backlog)	6
4. Хорошее	9
5. Замечания / nitpicks	9
6. Architectural divergence flagged	11
7. Open questions back to Alina (новые, после этого drop’a)	12
8. Risks Alina explicitly flagged	12
9. Конкретные действия для нас (Phase B implementation backlog)	12
10. Status update	13

Review — Alina’s Circuit 4 v3 single-final-root drop (2026-05-22)

Reviewer: Sergey Egorov (Pruvendo / bridge integration) **Date:** 2026-05-22

Scope — three sibling branches Alina pushed this morning that together close the live Circuit 4 loop end-to-end:

Repo	Branch	Tip
gosh-sh/acki-nacki-to-eth-bridge-halo2-circuits	circuit4-single-final-root	08d5fa9 cleanup (2026-05-22 09:32 +03)
gosh-sh/acki-nacki-to-eth-bridge-halo2-prover	full_bridge_flow_test_single_final_root	786a9f0 switch to single-root layout (2026-05-22 08:10 +03)
gosh-sh/acki-nacki	test_bridge_poseidon_dex (forked from poseidon_hex)	c248f16 W=128 + E2E margin drop (2026-05-21)

Reviewed against:

- docs/an_partner_questions_circuit4_2026-05-17.md (Q-C4-1..6 + concept).
- docs/an_partner_circuit4_alina_replies_2026-05-21.md (Alina’s prose replies + our de-coded layout).
- docs/circuit_4_open_questions.md (Phase A scaffold spec).
- contracts/ethereum/src/AckiNackiBridge.sol (current verifyEvent + _layerWindow[100] scaffold).
- contracts/ethereum/src/BridgeEventVerifier.sol + mock.

TL;DR

Это **очень хороший** drop. Алина закрыла все 6 наших Q-C4-* вопросов **и** дополнительно убрала самую дорогую часть Phase A scaffold’a на ETH-стороне: «multi-layer-hash zoo» — массив на $\text{MAX_LAYERS} \cdot W = 1280$ Fr public-input’ов, который контракт должен был передавать в proof.

Новый layout — **10 public Fr flat**, последний из которых (finalRoot) проверяется не в circuit’е, а off-circuit (т. е. в Solidity’е) на membership в текущем mirror’е layerWindows. Это снимает с нас 1280-элементную calldata на каждую операсию и переводит Circuit 4 verifier в практически тот же по форме API, что 1A/1B/2: `verify(vk, proof, pub[10])`.

Также — впервые — Circuit 4 VK перестал зависеть от W: commit fc63863 снёс cargo features w-8 / w-128 и NUM_LAYER_HASHES константу из circuit’a. **Один VK работает и для W=8 (тестовая среда), и для W=128 (production)**. Это огромный operational выигрыш для нас (не надо двух keygen ceremonies на Phase 9).

Конкретные нерешённые риски и stale documentation flagged ниже, но **функционально всё, что нужно для нашего Phase B (rewrite verifyEvent), есть**.

1. Что Алина изменила в circuit’e

1.1 Final public-input layout (bridge-event-prove-circuit, 10 slots)

bridge_event_prove_circuit.rs lines 4..27 (module head) + lines 113..123 (PUB_* constants) + lines 909..933 (push site):

Slot	PUB_* constant	Type	Источник
0	PUB_TOKEN_ID	uint32	BE-pack body[54..58)
1	PUB_AMOUNT	uint128	BE-pack body[38..54)
2	PUB_RECIPIENT_HI	uint80	BE-pack recipient_cell[2..12)
3	PUB_RECIPIENT_LO	uint80	BE-pack recipient_cell[12..22)
4	PUB_DST_CHAIN_ID	uint256	BE-pack body[6..38)
5	PUB_SENDER_ACC_FR	Fr	algebraic decode of sender cell bits[11..267)
6	PUB_DAPP_FR	Fr	destination account_dapp_id
7	PUB_ACC_FR	Fr	destination account_id
8	PUB_NULLIFIER	Fr	Poseidon(block_id, tokenId, amount, recipientHi, recipientLo, senderAccFr)
9	PUB_FINAL_ROOT	Fr	anchor root — checked off-circuit by verifier

TOTAL_PUBLIC_INPUTS = 10. Это совпадает 1-в-1 с нашим “финальным consolidated public-input layout” из an_partner_circuit4_alina_replies_2026-05-21.md строки 219-243, **минус sender-DappFr** (см. §1.4) и **минус 1280-элементный layerHashes хвост** (см. §1.2).

1.2 Single-anchor architecture (commit fc63863)

Это **главное архитектурное упрощение** drop’a. Цитата из commit message:

The bridge has no anonymization goal, so the previous design of exposing NUM_LAYER_HASHES = MAX_LAYERS * W public candidate layer hashes and privately selecting one via hash_choice_index is pure overhead. Replace it with a single PUB_FINAL_ROOT public slot that the verifier checks against its own known set of anchors off-circuit (mirrors the simpler gosh-dark-dex layout).

Что было: контракт передавал circuit’у 1280 candidate layer hashes, prover приватно выбирал один по private witness’у hash_choice_index. Anonymity feature.

Что стало: prover экспортирует **один** `final_root` как public input. Verifier (off-chain даемон в её prover-repo / Solidity у нас) проверяет, что этот `final_root` $\in$ `layerWindows[L].data[i]` для какого-то `L, i`.

Это именно тот вариант, который мы для себя зафиксировали в Q-C4-2: “anonymity не нужна, replay protection через nullifier”. Очень рад что она пришла к тому же выводу самостоятельно.

Что это даёт нам:

- Calldata к `proveWithdrawal` падает с $\sim 32 + 1280 \cdot 32 = 41$ KB до $\sim 32 + 10 \cdot 32 = 352$ B (плюс `proof_bytes`). На L1 это экономия $\sim \$8$ -12 на `withdrawal` по текущему gas price.
- ETH-side anchor check — линейный sweep по `MAX_LAYERS` $\times$ `W` (1280) bytes32 сравнений из storage. На `W=8` это 80, на `W=128` это 1280. На `W=128` это уже заметно ($\sim 50k$ gas в warm storage scenario), но всё равно дешевле чем передача 1280 elements + calldata gas; tight loop с break при первом match.
- **Circuit 4 VK больше не зависит от W** (cargo features w-8 / w-128 снесены в fc63863). Это **критично** для Phase 9 — нам нужна **одна** trusted setup ceremony на Circuit 4, а не две.

1.3 Sender binding (senderAccFr only, no senderDappFr)

Commit 8e271b8 “bind senderAccFr to BOC, drop senderDappFr” закрывает наш Q-C4-6 sub-question с обоснованной аргументацией:

The TVM address type (`std_addr$10`, см. `MsgAddrStd { anycast, workchain_id, address }` в `tvm-sdk/tvm_block/src/messages.rs`) carries **no** dApp-id field — only the workchain and the 256-bit `account_id`. The `dapp_id` lives in account state (`ShardAccount.dapp_id`), so binding it would require a separate `ShardAccount`-state Merkle proof — substantially more work, out of scope for v2.

И конкретный архитектурный совет на будущее:

If the destination side ever needs the sender’s dApp-id, the right architectural fix is for the AN bridge contract dev to add a `senderDappId` field directly to the `WithdrawalInitiated` event so it appears in the body cell BOC and can be parsed/bound the same way as `tokenId` / `dstChainId`.

Это **правильное** решение. Связь `sender`→`dapp` через `ShardAccount` requires ещё один Merkle path proof в circuit’е + ещё один SHA-256 chain, что `K` увеличило бы; а нам `senderDappFr` нужен только для прав-сценариев “верни мне средства если контракт-источник был чёрный”. Если когда-нибудь понадобится — пушим в AN bridge contract’е изменение события.

Помимо архитектурного “no”, Алина **алгебраически** декодирует `senderAccFr` прямо в circuit’е (см. §4.7 в `EVENT_LAYOUT_COMPARISON.md`, но он stale, реальный код — в `bridge_event_prove_circuit.rs` synthesize):

```
for j in 3..36:
    sender_bytes[j] = high3[j] * 32 + low5[j]    (range_check 3 / 5 bits)
for i in 0..32:
    account_id_byte[i] = low5[3+i] * 8 + high3[4+i]
senderAccFr = sum_i (account_id_byte[i] * 256^i)
```

Поскольку `sender` cell-bytes уже SHA-256-bound к body cell, а тот — к `ext_out` wrapper hash, `senderAccFr` malleability-free без отдельного orchestrator-supplied witness’а. Это лучше чем то, что мы изначально просили (мы готовы были на private witness + `bytes_to_fr` без алгебраической проверки).

1.4 Nullifier formula (Q-C4-2 closed)

`PUB_NULLIFIER` = Poseidon(`block_id_fr`, `tokenId`, `amount`, `recipientHi`, `recipientLo`, `senderAccFr`) — **6 элементов**, без `envelope_hash` (как и договаривались), без `senderDappFr` (см. §1.3).

Single-sponge call (hash_fix_len_array с RATE=2 → 3 absorb rounds + squeeze).

test_helpers::nullifier_native — native counterpart; MockProver-rect test_nullifier_recomputes_native доказывает что обе ветки эквивалентны. Хороший gate против case “circuit и native реализация разъехались”.

Replay-protection-семантика на ETH-стороне: require(!_nullifiers[bytes32(pubs[8])]) → _nullifiers[...] = true. Мы уже это запланировали.

1.5 Recipient split convention (Q-C4-1 закрыт)

RECIPIENT_HI_START = 2, RECIPIENT_HI_END = 12 и RECIPIENT_LO_START = 12, RECIPIENT_LO_END = 22 (см. bridge_event_prove_circuit.rs:188-191). Это **split α** — 10/10 BE-байт. ETH-side reassembly (закомментирована в самом circuit-файле, строки 187-188):

```
address recipient = address(uint160(
    (uint256(recipientHi) << 80) | uint256(recipientLo)
));
```

□ Confirmed split. Никакого ambiguity больше нет.

RECIPIENT_LEN_FIXED = 20 (комментарий в строке 125):

Ethereum addresses are 20 bytes. All 10 captured fixtures match this. To support variable lengths, see TODO §5.6 in EVENT_LAYOUT_COMPARISON.md.

□ Phase B → fixed 20 bytes, vary-length punted (как мы и договорились в Q-C4-4).

1.6 dstChainId в public (Q-C4-3 закрыт)

PUB_DST_CHAIN_ID = 4, parsed как uint256 BE от body[6..38). ETH-side ассертит pubs[4] == block.chainid. □

1.7 Circuit-shape highlights

- K = 19 (см. test_helpers::K)
- num_advice_per_phase = vec![16], lookup_bits = Some(18)
- num_instance_columns = 1
- Pipeline (13 шагов, перечислены в module head): SHA-256 → 3 child-hash equality links → field extraction + ABI prefix constrain (0x3c838959) → 4-cell refs_count constrain (1,2,0,0) → repr_hash_fr pack → ext_msg_leaf = Poseidon96(dappFr, accFr, repr_hash) → events Merkle proof → block_leaf = Poseidon96(block_id, envelope_hash, ext_out_root) → block Merkle proof → dense chain (MAX_CHAIN_LEN=11) → nullifier sponge → instance push.

Это **тот же pipeline**, что в gosh_dark_dex_halo2_circuit (DarkDexCircuitNew), что неоднократно подчёркивается в комментариях. Reuse — хорошо для аудита: chip’ы уже отревьюлены в docs/layer_hashes_circuit_audit.md.

2. Что Алина построила в prover-геро

2.1 Архитектура (по README)

```
GraphQL node → bridge-prover (1A + 2)          → proof_NNN.json
                                                    ← result_NNN.json
GraphQL node → bridge-event-private-witness-export
                → bridge-event-witness-builder
                → bridge-event-prove (Circuit 4) → proof_event_NNN.json
                                                    ← proof_event_NNN.result.json
                bridge-verifier (1A + 2 + 4 + state mirror)
```

3 binaries + verifier-daemon — все file-based, IPC через JSON в proofs/. **Verifier-daemon = модель Ethereum bridge contract’a** на стороне prover-repo (bridge-verifier-daemon/src/main.rs, цитирую комментарий из строки 552):

The verifier daemon models the future Ethereum bridge contract. It accepts a Circuit 4 proof only if the ... layer hashes baked into its public instances match the daemon’s *current* mirrored BridgeState.layer_windows[1..=MAX_LAYERS] byte-for-byte.

Это **именно та** state-mirror-семантика, которую мы должны реализовать в AckiNackiBridge.verifyEvent. См. §3 ниже — verifier daemon в её репо — **наша референс-имплементация**, по которой мы будем писать Solidity.

Ключевая часть anchor check в bridge-verifier-daemon (lines 757-768):

```
let current_hashes = state.flatten_layer_hashes();
debug_assert_eq!(current_hashes.len(), MAX_LAYERS * state.window_size);
let final_root_bytes: [u8; 32] = instances[9].to_repr();
let anchor_matched = current_hashes.iter().any(|h| *h == final_root_bytes);
if !anchor_matched {
    // reject with anchor_mismatch
    return;
}
// then run halo2 verify
let proof_valid = event_verifier::verify_event_proof(key_manager, &proof_bytes, &instances);
```

Это **off-circuit anchor membership check**. Solidity-эквивалент: nested loop по layerWindows[L].data[i] с early break.

2.2 BridgeState mirror

bridge-prover-lib/src/bridge_state.rs (359 lines added) — это off-chain twin Ethereum-контрактной storage’и:

- layer_windows: [HistoryWindow; MAX_LAYERS] — на каждый layer $L \in 1..=MAX_LAYERS$ хранится data: [Fr32; W] + heights: [u64; W] + head: usize. Это **ring buffer** на W элементов per-layer.
- append_bundle(new_layer_hashes, block_height, block_seq_no, bk_hash) — атомарное добавление layer hashes из одного Circuit-2 proof’a. Filter out all-zero entries (means “prover left that layer unset”).
- stored_last_seen_block_seq_no, stored_last_seen_block_height, stored_bk_set_commitment — те же 3 anchor’a, что в AckiNackiBridge.
- Monotonicity check: refuse to append если $bundle\ block_seq_no \leq stored$ (см. verifier-daemon line 405-413).

2.3 bootstrap.rs — genesis seed

Cold-start path: prover-daemon на первом key-block’e пишет state/bootstrap_seed.json, verifier-daemon его читает один раз при старте. Это **эквивалент конструктора** Ethereum-контракта, получающего initial GlobalHistoryData (комментарий в verifier-daemon строки 113-127):

This mirrors the on-chain contract receiving its genesis GlobalHistoryData via constructor arguments at deployment.

Implication для нас: **AckiNackiBridge.constructor должен принимать BootstrapSeed-эквивалент** — initial layer_hashes + block_seq_no + block_height + bk_set_commitment. Сейчас наш конструктор этого не делает; в verifyBlock мы bootstrap’имся “лениво” на первом block’e. Алина’s подход точнее (соответствует production semantics) — обсудим в §6.

2.4 E2E test (orchestrator)

acki-nacki/tests/exchange/generate_withdrawals_with_live_event_proving.py:

1. Deploy multisig wallet, fund with ECC[2]
2. Trigger initiateWithdrawal via TokenBridge
3. Polling GraphQL для извлечения block_seq_no, block_height, envelope_hash, account_dapp_id, account_id
4. Compute thinned_kb_seq = ((event_seq // (W·P)) + 1) · W · P
5. Wait until verifier state advances to thinned_kb_seq
6. bridge-event-private-witness-export → bridge-event-witness-builder → bridge-event-prove --fixture ...
7. Wait for .result.json from verifier daemon
8. Assert verified == true && anchor_matched == true

Measured: ~**11:18 wall-clock** end-to-end на 5-node local devnet (W=128, P=4). Bundle prove ≈ 4 min, Circuit 4 prove ≈ 2.5 min, plus 2 bundle waits.

2.5 Verifier-daemon stats

После каждого event proof пишется proof_event_NNNNNN.result.json с такими полями (см. bridge-verifier-daemon/src/main.rs lines 590-617):

```
{
  "schema_version": 1,
  "seq_no": 0,
  "verified": true,
  "anchor_matched": true,
  "proof_valid": true,
  "prover_self_verified": true,
  "verified_at_block_height": 1024,
  "verified_at_block_seq_no": 1024,
  "event_public_instances_hex": ["...", "...", ...],
  "error": null
}
```

Это **золотая** test-vector форма — для нашего Foundry test'a на новый verifyEvent мы можем взять proof_event_*.json + bridge_state snapshot и прогнать сначала off-chain через её verifier-daemon (чтобы убедиться что proof valid в её референсной интерпретации), а потом on-chain через наш Solidity и сравнить anchor-match outcome.

3. Что меняется у нас на ETH-стороне (Phase B implementation backlog)

3.1 BridgeEventVerifier.sol — hard ABI break

Текущий scaffold ожидает **103** public Fr: [tokenId, dappFr, accFr, layerHashes[0..100]]. Новый layout — **10** public Fr.

Действия:

- ☐ Удалить IBridgeEventVerifier/BridgeEventVerifier scaffold с 103-input ABI.
- ☐ Создать новый адаптер на 10-input Groth16 verifier (когда Phase 9 wrapper landed) или временно — на mock, идентичный по shape.
- ☐ BridgeEventGroth16VerifierGenerated.sol — будет регенерирован Алиной заново под 10-input shape после Phase 9. Сейчас можно жить на mock.

3.2 AckiNackiBridge.verifyEvent — переписать целиком

Текущая signature: verifyEvent(bytes calldata proof, uint64 tokenId). Новая (по аналогии с Алининой verifier-daemon):

```
function verifyEvent(
    bytes calldata proof,
    uint256[10] calldata pubs
) external {
    // 1. Структурные проверки слотов
    require(pubs[4] == block.chainid, "BadDstChainId"); // dstChainId
    require(uint160(pubs[0]) <= type(uint32).max, "BadTokenId"); // tokenId
    // ... (остальные range-check'и пер `PUB_*` константам)

    // 2. Replay protection
    bytes32 nullifier = bytes32(pubs[8]);
    require(!_nullifiers[nullifier], "Replay");

    // 3. Anchor membership (off-circuit check)
    bytes32 finalRoot = bytes32(pubs[9]);
    require(_anchorMatches(finalRoot), "AnchorMismatch");

    // 4. Cryptographic verify
    require(eventVerifier.verifyProof(proof, pubs), "InvalidProof");

    // 5. Effects
    _nullifiers[nullifier] = true;
    address recipient = address(uint160(
        (uint256(pubs[2]) << 80) | uint256(pubs[3])
    ));
    uint128 amount = uint128(pubs[1]);
    uint64 tokenId = uint64(pubs[0]);
    _transferOut(recipient, amount, tokenId);

    emit WithdrawalProven(nullifier, recipient, amount, tokenId);
}

function _anchorMatches(bytes32 finalRoot) internal view returns (bool) {
    for (uint8 L = 1; L <= MAX_LAYERS; L++) {
        bytes32[] storage data = layerWindows[L].data;
        uint256 n = data.length;
        for (uint256 i = 0; i < n; i++) {
            if (data[i] == finalRoot) return true;
        }
    }
    return false;
}
```

Замечания:

1. **Recipient reconstruction** — confirmed split α . ETH-сторона `address(uint160((uint256(pubs[2]) << 80) | uint256(pubs[3])))`.
2. **Nullifier mapping** — `mapping(bytes32 => bool) _nullifiers`.
3. **dstChainId** — uint256 slot, акцепт == `block.chainid`.
4. **pubs[5..7]** — `senderAccFr, dappFr, accFr` — контракт **не валидирует** (это просто инстансы proof'a, нужны для VK match), но **может** залогировать в event для audit-trail.
5. **_transferOut(...)** — отдельный internal: `tokenId == 0` → `recipient.call{value: amount}`, иначе → `tokens[tokenId].transfer(recipient, amount)`. Reentrancy-guard как везде у

нас.

3.3 layerWindows[MAX_LAYERS] mirror — переделать _layerWindow[100] ring scaffold

Сейчас в AckiNackiBridge.sol (Phase A scaffold, 2026-05-17):

```
bytes32[100] private _layerWindow;  
uint8 private _layerWindowHead;
```

Это **плоское** ring-buffer на 100 элементов. Под новый single-final-root architecture нужна Alina-mirror structure (bridge_prover_lib::bridge_state::HistoryWindow):

```
struct HistoryWindow {  
    bytes32[W_SIZE] data;  
    uint64[W_SIZE] heights;  
    uint16 head;  
}  
  
mapping(uint8 => HistoryWindow) public layerWindows; // L ∈ 1..=MAX_LAYERS  
  
function _appendLayer(uint8 L, bytes32 root, uint64 height) internal {  
    HistoryWindow storage w = layerWindows[L];  
    w.data[w.head] = root;  
    w.heights[w.head] = height;  
    w.head = uint16((w.head + 1) % W_SIZE);  
}
```

Где W_SIZE — compile-time константа. **Проблема:** W=8 vs W=128 — это **разный storage layout**, выбор делается на deploy-time. Под наш план: одна deploy-script переменная AN_HISTORY_WINDOW_SIZE, по умолчанию 128. Можно сделать динамической через bytes32[] (без fixed-size), но это лишний gas на bound-check'ах. **Рекомендация:** fixed W = 128 для production, отдельный deploy для testnet с W = 8.

verifyBlock (Phase 4.1) должен **дополнительно** делать _appendLayer(L, layer_hash, block_height) для каждого non-zero layer hash в Circuit 2 proof'e. Cross-check: verifier-daemon делает то же самое (bridge-verifier-daemon/src/main.rs:419-446).

3.4 MockBridgeEventVerifier + tests — переписать

- 16 тестов в AckiNackiBridgeVerifyEvent.t.sol сейчас гонятся под 103-input mock. **Все** надо адаптировать: ABI signature, layout строгого mode, expected pubs.
- Strict-mode mock: pin'ит expected_pubs[10] + sanity-check'ит final_root ∈ expected_layer_anchors (моделирует off-circuit anchor check).
- Новые тесты, которые **обязательно** добавить:
 - **anchor mismatch:** final_root не из current layerWindows → revert AnchorMismatch.
 - **replay:** один и тот же nullifier → второй вызов revert.
 - **stale anchor:** final_root был валиден 100 блоков назад, но сейчас ring rotated → revert.
 - **dstChainId != block.chainid** → revert.
 - **recipient reconstruction round-trip:** hi/lo → address → assert == known recipient.

3.5 Constructor — принять bootstrap seed

Сейчас наш AckiNackiBridge.constructor (5 args) не принимает initial layer hashes. По Алининой verifier-daemon семантике, контракт должен **либо** запрашивать seed как часть deploy args, **либо** иметь отдельный owner-only initialize(BootstrapSeed) который вызывается один раз.

Рекомендация: добавить `initialize(BootstrapSeed)` с `onlyOwner + initialOnce` modifier'ом, отделить `deploy` от `seed`'инга. Это даёт нам: 1. Возможность `deploy`'ить контракт **до** того, как AN-сеть готова (тестовый стенд, `fork tests`). 2. Возможность `re-seed` после `major upgrade`'а AN-сети (например, при переходе с `W=8` на `W=128` — тогда нужен полный `rebuild storage`'и).

3.6 Documentation updates

- `docs/circuit_4_open_questions.md` — закрыть Q-C4-1..6, пометить новый набор `open question`'ов (см. §7 ниже).
 - `docs/four_circuit_architecture.md` — обновить таблицу `per-circuit public inputs` для Circuit 4 (103 → 10).
 - `docs/an_partner_integration_plan.md` Phase A scaffold секцию — пометить как `retiring`, замена в Phase B.
 - `AGENTS.md` — обновить `test count` после переписывания `AckNackBridgeVerifyEvent.t.sol`.
-

4. Хорошее

1. **Closure of all our Q-C4-1..6 вопросов** в одной серии `commit`'ов, с обоснованием. Все наши декодированные ответы (`docs/an_partner_circuit4_alina_replies_2026-05-21.md` строки 219-243) проявились в коде 1-в-1, кроме намеренного `senderDappFr-drop` с хорошим архитектурным обоснованием.
 2. **Single-final-root design** — на голову лучше нашей Phase A scaffold. На production `W=128` это \$8-12/withdrawal калькачастью только `calldata`, плюс Circuit 4 VK W-agnostic (одна `trusted setup ceremony` вместо двух).
 3. **Algebraic decode of senderAccFr** — лучше `private witness`'а; добавляет `malleability-resistance` без `extra chip`'ов.
 4. **Verifier-daemon = reference impl** — у нас есть `workable reference` на ~800 LOC Rust'а, по которой можно побайтно сверять Solidity behavior. `Schema-version` field в IPC формате — позволит без `headache` мигрировать формат.
 5. **hash_fix_len_array для nullifier'a + parallel native test** (`test_nullifier_recomputes_natively`) — `gate-against drift` между `circuit` и `native implementation`. Хороший `pattern`.
 6. **K=19, advice=16, lookup_bits=18** — conservative parameters, не оптимизированы (как у `DarkDexCircuitNew K=14`). Это **хорошо** для первой версии: `tightening parameters` — это последняя миля перед `mainnet`, а сейчас важнее `correctness`.
 7. **load_first_withdrawal_embedded** через `include_str!("../withdrawals.txt")` — fixtures baked в `binary`. Downstream crate'ы (`bridge-prover-lib::keys::ensure_event_keys`) не нуждаются в `filesystem access` для `keygen`. Хороший `pattern` для CI и `reproducibility`.
 8. **Schema-versioning IPC** (`schema_version: u32` в `EventProofFile` + `EventProofResult`, `daemon hard-fails` на `mismatch`). Это **именно** то, чего обычно не хватает в `first-cut prototype`'ах. Готово к долгой эксплуатации.
-

5. Замечания / nitpicks

5.1 Stale documentation across both repos

`fc63863` снёс многое из `circuit`'а, но **не** пробежал по `docs`'ам. Конкретно `stale`:

- `acki-nacki-to-eth-bridge-halo2-circuits/README.md`:
 - Секция **“The Five Circuits”** строка 56 — Circuit 4 column всё ещё пишет `[tokenId, dapp_fr, acc_fr, layer_hashes[0..MAX_LAYERS*W]]`. Должно быть 10 flat slots.
 - Секция **“Why Circuit 4's VK depends on W”** — после `fc63863` это **неверно**. VK теперь W-agnostic.
 - Секция **“Ethereum Contract”** `proveWithdrawal` Solidity-набросок строки 80-110 — описывает `pub4 = [tokenId, dappFr, accFr, layerHashes[0..NUM_LAYER_HASHES]]`

которое строится в контракте из layerWindows. **Stale.** Должно быть pub4 это просто 10 input'ов от caller'a

* off-circuit anchor membership check.

- acki-nacki-to-eth-bridge-halo2-circuits/bridge-event-prove-circuit/src/EVENT_LAYOUT_COMPARISON.md
 - **§4.5 “Multi-layer-hash choice”** — описывает дропнутую логику hash_choice_index + select_from_idx.
 - **§4.6 “Public instance layout (column 0, v2)”** — указывает TOTAL_PUBLIC_INPUTS = 9 + NUM_LAYER_HASHES. Должно быть просто 10.
- acki-nacki-to-eth-bridge-halo2-prover/README.md:
 - **Step 0** — “Circuit 4 feature flag | bridge-event-prove-circuit/w-128”. **Stale:** feature flag снесён в fc63863.
 - **Step 3** keygen table — “Bridge Event Prover (K=19) | event_vk.bin, event_pk.bin | ... must run before bridge-verifier”. Это всё ещё верно, но комментарий “w-8 Cargo feature exists for Circuit 4 MockProver tests” тоже stale.
 - **Step 0 sibling-repo table** ссылается на circuit4-w-parameterized, которая суперседнута circuit4-single-final-root.

Это **не** функциональные баги, но downstream consumers (нам, кому-то другому) будут спотыкаться. Stop-the-bleed PR — отдельный doc-only коммит с synсом всех 3 docs'ов. Тривиально, но критично для onboarding'a.

5.2 Anchor sweep cost on W=128

Внутри _anchorMatches на ETH-стороне — линейный sweep по MAX_LAYERS × W = 1280 slot'ам. Worst case (anchor в последнем layer'e, последнем slot'e): 1280 × ~2,100 gas (cold SLOAD) = 2.7M gas, что превышает target 1M/withdrawal.

Mitigations: - **Storage layout:** складывать все layer hashes в один bytes32[1280] array (а не mapping(uint8 => bytes32[W])). Снизит storage warm-up на 1 SLOAD после первого hit'a на адрес slot'a. - **Bloom filter / hash table:** добавить mapping(bytes32 => bool) _activeAnchors. Maintain'ить его в _appendLayer (delete old, insert new). O(1) membership check вместо O(1280). **Strongly recommended** для W=128. - **L-hint от prover'a:** добавить **public input** или отдельный calldata argument anchorLayer: uint8 — тогда контракт проверяет только layerWindows[anchorLayer]. data[i] == final_root. Снижает worst-case до 128 SLOADs. Это leak'ает на каком layer'e был block (минимальный info-leak; у нас anonymity не цель).

Рекомендация: реализовать (b) или (c). Это **наша** оптимизация, не Алинина — она сделала off-circuit anchor check как principle, реализация check'a — наш score.

5.3 Bootstrap seed format не stable

bridge-prover-lib::bootstrap::BootstrapSeed ходит через state/bootstrap_seed.json без schema_version. Если Алина изменит формат — verifier-daemon (и наш контракт) сломается silently. Это inconsistent с EventProofFile/Result, которые schema-versioned.

Рекомендация для Алины: добавить schema_version: u32 в BootstrapSeed. Single line change, потом нам не больно.

5.4 No senderDappFr — accept, but document the gap

§1.3 уже выше — Алина обосновала почему. Но у нас в docs/an_partner_circuit4_alina_replies_2026-05-21.md финальный layout всё ещё показывает senderDappFr в slot[5]. Нужно обновить тот док, чтобы будущий читатель не искал несуществующее поле.

5.5 Recipient cell RECIPIENT_LEN_FIXED = 20 hardcoded

Если AN сторона когда-нибудь поддержит non-ETH chain (Solana, Aptos, Sui) с recipient'ом длиннее 20 байт — придётся менять circuit + VK + ceremony. Это **не блокер** сейчас; multi-chain — Phase 10+.

Рекомендация: в BridgeEventVerifier адаптере добавить константу `RECIPIENT_LEN = 20` с комментарием, что это **must** соответствовать `bridge-event-prove-circuit:: RECIPIENT_LEN_FIXED`. CI guard на это (single-line grep в pre-push) — дешево.

5.6 Verifier-daemon's _nullifiers enforcement не реализован

Из комментария verifier-daemon (lines 564-570):

the daemon currently does not yet enforce a proven[] map against the nullifier slot — that, plus recipient binding to the on-chain msg.sender, are the remaining post-verification TBD items.

ОК — это **наша** работа на ETH-стороне (§3.2). Алина просто не реализовала replay protection в reference impl'e, потому что её фокус — прувер. Но если она хочет full E2E: рекомендую добавить `nullifier_seen: HashSet<u8; 32>` в её BridgeState тоже, чтобы её verifier-daemon полностью моделировал Solidity-семантику. Можно flag'ом — "replay-protection-mode".

5.7 verifyEvent not yet wired into bundle flow

Verifier daemon сейчас обрабатывает `proof_event_*.json` независимо от `proof_NNN.json`. На Solidity-стороне (у нас) event verification и bundle verification — **разные функции** (`verifyEvent` vs `verifyBlock`), так что это ОК. Но **порядок**: event proof admissible только **после** того, как bundle, покрывающий его key block, был relayed. Verifier-daemon **сейчас** этого не enforce'ит явно — он просто полагается на `flatten_layer_hashes()` returning the current state. Нам надо это enforce'ить **в Solidity** (implicit via anchor membership check — если `layerWindows` ещё не содержат соответствующий root, proof just won't anchor-match → revert).

ОК — implicit enforcement через anchor check работает. Но **стоит** добавить explicit test'кейс: event proof, сгенерированный для block N, проверяется ДО того, как bundle для block N был relayed → expected revert AnchorMismatch.

6. Architectural divergence flagged

6.1 Bootstrap из конструктора vs lazy-init

Алина seed'ит через `bootstrap_seed.json` который verifier-daemon читает один раз на старте. У нас в Solidity сейчас lazy init — первый `verifyBlock` инициализирует.

Проблема с lazy init: если первый `verifyBlock` **неправильный** (wrong AN block, wrong `bk_set`), мы зацементируем мусор и не сможем recover без redeploy.

Решение: `initialize(BootstrapSeed)` от owner'а **один раз**, дальше lazy-init блокирован. Алинина модель это уже подразумевает (`verifier-daemon bootstrap_seed.json = constructor args`).

6.2 _layerWindow[100] Phase A → layerWindows[MAX_LAYERS] Phase B

Это **полная** pekstr storage'и. Не migration через проху- upgrade, а fresh deploy с новым layout'ом. Учитывая что Circuit 4 VK тоже меняется (а у нас сейчас 103-input mock anyway), это ОК — Phase A scaffold можно просто снести.

6.3 Per-layer rolling window vs global layer hashes

Сейчас наш `verifyBlock` хранит `storedLayerHashes[10]` — просто 10 последних layer hash'ей. Это **не** соответствует тому, что Алина хранит (HistoryWindow per-layer с W ring buffer entries). Её structure правильнее: у каждого layer'а свой ring.

7. Open questions back to Alina (новые, после этого drop'a)

1. **W=8 vs W=128 cohabitation.** Circuit 4 VK теперь W-agnostic — но что с Circuit 2 / 1A? Они зависят от `MAX_CHAIN_LEN = 11` (одинаковый для всех W?). Нам надо подтвердить: **один** keygen на 1A/2 покрывает и W=8 (test), и W=128 (prod)?
2. **Storage layout для W.** Production будет fixed W=128 или dynamic? Если dynamic — какой schema migration path при W change?
3. **MAX_LAYERS.** Сейчас 10 в circuit'e (`PUB_NULLIFIER + PUB_FINAL_ROOT` order suggests `MAX_LAYERS = 10?`). Это жёстко прибито или есть feature flag? Если жёстко — подтверждаем компилируем under `MAX_LAYERS=10`.
4. **Bootstrap seed schema.** Можно ли добавить `schema_version: u32` в `BootstrapSeed`? Тривиальное изменение, спасёт от silent breakage.
5. **Replay protection в BridgeState.** Имеет смысл добавить `nullifiers: HashSet<[u8; 32]>` в её state и `proven[] enforcement` в `verifier-daemon`, чтобы full E2E test покрывал reject-on-replay scenario? (Нам всё равно нужно — у нас есть в Solidity, но можно убедиться что её reference impl behaviour аргит.)
6. **Anchor sweep optimization.** Мы будем строить `mapping(bytes32 => bool) _activeAnchors` для O(1) lookup. У неё есть мнение о trade-off? (Storage cost vs warm-SLOAD-loop cost; Geth gas constants.)
7. **senderAccFr в Poseidon без workchain constraint.** В коде §4.7 Алина пишет: "Routing bits (`std_addr$10` tag / anycast / workchain) are NOT constrained here — see §5 "constrain d2 and tighten entries[3] bit-prefix"". Это значит prover может подсунуть sender cell с **любым** workchain'ом или anycast'ом. На bridge-сценарии где sender — always TIP-3 контракт на workchain 0, это acceptable. Но **flag**: если AN когда-нибудь захочет multi-workchain bridge, это constraint надо добавить.

8. Risks Alina explicitly flagged

Из её message'a:

1. **Untested on shellnet** — works on local 5-node devnet (~3 source blocks/s, W=128). Shellnet:
 - возможно сейчас W=128 vs её compile-time check (если так — OK);
 - Саша запустил какие-то fix'ы которые **могут** опять сломать node-block-client;
 - её orchestrator работает **только** при старте с `seq_no = 0` (первый key-block). Long-running shellnet → требует bootstrap from snapshot, которого сейчас нет.
2. **THINNING_FACTOR_P = 4** — bundle width $W \cdot P = 512$ блоков ≈ 4 min на 3 b/s devnet. На shellnet rate может быть другой → timeout'ы в orchestrator'e могут трипать.

Наш план: не пытаемся гонять её E2E на shellnet'e. Локальный devnet — sufficient evidence для того чтобы мы начали Phase B Solidity rewrite. Shellnet validation — после того, как Саша стабилизирует node-block-client.

9. Конкретные действия для нас (Phase B implementation backlog)

#	Действие	Effort
B-1	Snapshot Alina's branches в dist/ для офлайн-чтения и pin'нуть в Cargo.lock через git rev	0.5 ч

#	Действие	Effort
B-2	Сгенерировать local devnet test artefacts (event_vk.bin + ≥ 3 proof_event_*.json + соответствующие bridge_state.json) для Foundry fixtures	1 day (1 × E2E $\approx$ 12 min)
B-3	Переписать BridgeEventVerifier.sol с 103 $\rightarrow$ 10 public inputs	1 day
B-4	Переписать AckiNackiBridge.verifyEvent (§3.2) + добавить initialize(BootstrapSeed) (§3.5) + layerWindows[MAX_LAYERS] rewrite (§3.3)	2 days
B-5	Anchor lookup optimization — mapping(bytes32 $\Rightarrow$ bool) _activeAnchors (§5.2)	0.5 day
B-6	Переписать MockBridgeEventVerifier + 16 tests, добавить 5 новых negative tests (§3.4)	1.5 days
B-7	Real-proof Foundry fixture test, прогон proof_event_*.json через новый contract $\rightarrow$ сравнить anchor-match outcome с bridge_prover_lib::bridge_state::flatten_layer_hashes()	0.5 day
B-8	Doc updates: close Q-C4-* in circuit_4_open_questions.md, обновить four_circuit_architecture.md, an_partner_integration_plan.md, AGENTS.md	0.5 day
B-9	Открыть отдельный мини-PR в Алинины repo с doc fix'ами (§5.1 stale documentation)	0.5 day

Итого Phase B: $\sim$ 7-8 рабочих дней с тестами и доками, после получения local devnet fixtures (B-2).

Phase 9 (trusted setup ceremony per circuit) остаётся отдельной дорожкой — не блокирует Phase B так как у нас на ETH-стороне mock-verifier для Circuit 4 до landing'a реального gnark wrapper'a.

10. Status update

- **Все 6 наших Q-C4-* — закрыты** в этом дроп'е (с одной заметкой про намеренный senderDappFr-drop и архитектурным планом B на будущее).
- **3 новых open question** к Алине (см. §7 #1-7).

- **3 stale docs** в её репо (см. §5.1) — рекомендую отдельный doc-only PR от нас как goodwill (получится 5-10 строк changes, никаких code conflicts).
- **Phase A scaffold (_layerWindow[100], 103-input mock verifier) — retiring.** Phase B implementation начинается.